SOFTWARE REQUIREMENTS SPECIFICATION

CampusMind AI

University Knowledge Assistant Using RAG,
Hybrid Search, and Redis Caching

Document	Software Requirements Specification (SRS)
Version	1.2
Project	CampusMind AI
Document status	Project requirements baseline

Prepared as the functional and technical baseline for implementation.

Document Control

Item	Description
Project name	CampusMind AI
Document title	Software Requirements Specification
Version	1.2
Purpose	Define the agreed scope, requirements, architecture, technology choices, and workflows.
Guiding principle	Keep the implementation simple, clean, understandable, and competitive. Avoid unnecessary complexity.

Table of Contents

- 1. Introduction
- 2. User Roles and Permissions
- 3. System Pages
- 4. Technology Stack
- 5. System Architecture
- 6. Database and Storage Design
- 7. Functional Requirements
- 8. Main System Workflows
- 9. Non-Functional Requirements
- 10. Testing and Evaluation
- 11. Initial Project Structure
- 12. Constraints and Assumptions
- 13. Future Improvements
- 14. Expected Deliverables
- 15. Development Guidelines

1. Introduction

1.1 Purpose

This document specifies the requirements for CampusMind AI, a university knowledge assistant that helps users ask questions about university documents and receive answers grounded in retrieved source material. It defines what the system must do, who can use it, the technologies selected, and the main implementation workflows.

1.2 Project Overview

CampusMind AI uses Retrieval-Augmented Generation (RAG). It extracts text from uploaded university documents, divides the text into chunks, creates embeddings, retrieves relevant passages, and sends those passages to an external Large Language Model (LLM) to produce an answer with source references.

The retrieval layer combines Qdrant vector search and BM25 keyword search. Reciprocal Rank Fusion (RRF) combines their ranked results. Redis caches successful answers so repeated questions can be answered faster without repeating retrieval and LLM generation.

1.3 Objectives

- Provide a simple chat interface for students and staff to ask questions about university information.
- Generate answers grounded in available university documents and show source references.
- Support document upload and processing by administrators.
- Improve retrieval by combining semantic vector search with BM25 keyword search.
- Reduce repeated processing and LLM calls through Redis answer caching.
- Allow administrators to monitor documents and cache activity.
- Keep the application understandable, maintainable, and suitable for demonstration and evaluation.

1.4 Scope

- User registration, login, and role-based access.
- Public home page and authenticated AI chat.
- Administrative dashboard, document management, and Redis cache analytics.
- PDF, DOCX, and TXT document ingestion.
- Fixed-size, recursive, and semantic chunking methods, each supporting configurable overlap.
- Sentence Transformer embeddings, Qdrant vector search, BM25 keyword search, and RRF hybrid retrieval.
- LLM answer generation grounded in retrieved passages and accompanied by source references.
- Redis cache-aside answer caching and basic cache performance metrics.
- MySQL storage for application data and chat history.

Out of scope for the initial version: microservices, distributed task queues, complex multi-agent workflows, model training, and advanced reranking. These are not needed for the first working system.

2. User Roles and Permissions

Role	Permissions
Guest	View the public home page; open the login/register page. Must log in before using chat.
Student	Use AI chat; view and manage their own chat history as supported by the interface. Cannot access admin pages.
Staff	Use AI chat. Cannot access admin pages unless explicitly assigned the admin role.
Admin	Access the admin dashboard, document management, cache analytics, and AI chat. Manage documents and view system-level information.

Authorization must be enforced by the backend. Hiding an admin link in the frontend is not sufficient protection. Every protected API endpoint must verify the authenticated user's role.

3. System Pages

Page	Access	Main purpose
1. Home Page	Public	The same public landing page is visible before and after login. Introduces the system and provides an Ask a Question / Chat action.
2. Login/Register	Public	Allows users to create an account and log in.
3. AI Chat	Authenticated users	Allows students, staff, and admins to ask questions and receive grounded answers with sources.
4. Admin Dashboard	Admin only	Shows a concise overview of system activity and document/cache status.
5. Document Management	Admin only	Allows admins to upload, view, update, and delete supported documents and see processing status.
6. Redis Cache Analytics	Admin only	Shows cache hits, misses, hit rate, latency, memory usage where available, and LLM calls avoided.

3.1 Navigation Rules

- Everyone can view the same public Home Page.
- If a guest clicks Ask a Question / Chat, the system redirects them to Login/Register.
- After successful login, students and staff are directed to AI Chat.
- After successful login, admins are directed to the Admin Dashboard by default; AI Chat remains available through admin navigation.
- Students and staff cannot open administrative pages or endpoints.
- Admin navigation includes Dashboard, Home, AI Chat, Documents, Cache Analytics, and Logout.

4. Technology Stack

Area	Selected technology	Purpose
Frontend	React.js + Tailwind CSS	Build the web interface and responsive page styling.
Backend	Python + FastAPI	Provide API endpoints, authentication, ingestion, retrieval, caching, and chat logic.
Relational database	MySQL	Store users, documents, chat sessions, messages, and structured application data.
Vector database	Qdrant	Store chunk embeddings and metadata; perform vector similarity search.
Vector index	Qdrant HNSW	Speed up approximate nearest-neighbor vector search.
Keyword retrieval	BM25	Find exact terms, phrases, names, codes, and identifiers in document chunks.
Hybrid fusion	Reciprocal Rank Fusion (RRF)	Combine Qdrant and BM25 ranked results.
Embeddings	Sentence Transformers: BAAI/bge-small-en-v1.5	Convert document chunks and questions into vectors.
Cache	Redis + redis-py	Cache successful answers and their source references.
LLM	External LLM API	Generate answers using retrieved document context.
Document extraction	PyMuPDF, python-docx, plain text	Extract text from PDF, DOCX, and TXT files.
Chunking	LangChain Text Splitters + simple custom Python functions	Implement recursive splitting and the selected custom chunking methods.
Authentication	JWT + password hashing	Authenticate users and protect API routes.
Local services	Docker Desktop for Qdrant and Redis	Run supporting services locally during development.
Testing / tools	Postman, Git, GitHub, MySQL Workbench	Test APIs, manage code, and inspect the relational database.

The backend will be one modular FastAPI application. The project will not be split into microservices for the initial version.

5. System Architecture

The system is organized into a frontend, one FastAPI backend, and supporting data services. The backend coordinates authentication, document ingestion, retrieval, LLM calls, Redis caching, and chat history.

5.1 Main Components

- **React frontend:** presents the public pages, authentication screens, AI chat, and admin pages.
- **FastAPI backend:** validates requests, enforces roles, coordinates the RAG pipeline, and communicates with storage services.
- **MySQL:** stores structured user, document, and conversation records.
- **Qdrant:** stores vectors and chunk payloads for semantic retrieval.
- **BM25 index:** supports keyword-based retrieval over the same document chunks.
- **Redis:** stores reusable successful answers and source references.
- **Embedding model:** converts questions and chunks into vectors.
- **External LLM:** generates the final response from the question and retrieved context.

5.2 Retrieval Flow

Question → Redis check → (on cache miss) question embedding → Qdrant vector search + BM25 keyword search → RRF → evidence check → LLM answer with sources → Redis cache.

Redis must be checked before question embedding, vector retrieval, BM25 retrieval, fusion, or LLM generation. A cache hit skips those expensive steps.

6. Database and Storage Design

6.1 MySQL — Structured Application Data

MySQL stores application records rather than embedding vectors. The initial schema should remain small and include the following main tables:

Table	Important fields / purpose
users	id, name, email, password_hash, role, created_at
documents	id, filename, file_path, uploader_id, upload_date, status, chunk_count
chat_sessions	id, user_id, title, created_at
messages	id, session_id, role, content, timestamp, source references

Add indexes to frequently searched or joined columns when useful, such as users.email, chat_sessions.user_id, and messages.session_id. Avoid creating indexes on every column; add them based on actual query needs.

6.2 Qdrant — Vector Storage and Indexing

Each stored Qdrant point represents a document chunk and contains a unique point ID, an embedding vector, and a payload. The payload stores the chunk text and metadata such as documentId, documentName, chunkId, chunkIndex, and chunkingMethod. Include pageNumber when it is available.

HNSW indexing: Qdrant uses an HNSW index to make vector similarity search efficient. Use reasonable default settings initially. Tune parameters only if measured performance shows a need.

Payload indexes: create indexes for metadata fields that are frequently used in filters, such as documentId or department if department-based filtering is implemented. Do not add unnecessary payload indexes.

Use the correct embedding dimension and cosine similarity for the selected embedding model. Run Qdrant locally through Docker during development and persist its data using a Docker volume.

6.3 BM25 — Keyword Indexing

BM25 is a keyword-ranking algorithm, not a database. It indexes the text of the same chunks stored in Qdrant and retrieves chunks that match important query terms. It complements vector search for exact phrases, course codes, names, dates, and identifiers.

Keep the BM25 implementation simple using a suitable Python library. The BM25 index must be synchronized when documents are added, updated, deleted, or reindexed.

6.4 Redis — Answer Cache

Redis stores the final successful answer together with its source references. Use the cache-aside pattern: the backend checks Redis first; on a miss it runs the RAG pipeline and stores the successful result.

Normalize the question and create a SHA-256 hash using Python hashlib. The cache key should include a namespace, knowledge-base version, access scope, and question hash. Example: `rag:answer:public:v12:<question_hash>`.

Use an initial configurable TTL of 30 minutes (1800 seconds). When documents change, update the knowledge-base version or invalidate affected cache entries. Cache only successful grounded answers; do not cache failed API calls

or errors as successful answers.

If Redis is unavailable, the application should continue through the normal RAG pipeline rather than failing the user's question.

6.5 Original Documents

Store uploaded original files in local application storage for the initial version. Keep the file path and document metadata in MySQL. Apply file type and size validation before processing.

7. Functional Requirements

ID	Requirement	Description
FR-01	Public home page	The system shall provide a public home page accessible without login.
FR-02	User registration	The system shall allow a user to register with the required account details and securely store a password hash.
FR-03	User login	The system shall authenticate users and issue a JWT for protected requests.
FR-04	Role authorization	The system shall enforce student, staff, and admin permissions on protected backend routes.
FR-05	Chat access	The system shall require authentication before a user can access AI chat.
FR-06	Chat questions	The system shall accept a user's question and return an answer with source references when supported by available documents.
FR-07	Chat history	The system shall store chat sessions and messages in MySQL and associate them with the correct user.
FR-08	Document upload	The system shall allow admins to upload PDF, DOCX, and TXT documents.
FR-09	File validation	The system shall validate supported file types and configured file-size limits.
FR-10	Text extraction	The system shall extract text from uploaded documents using the appropriate extraction library.
FR-11	Text cleaning	The system shall clean extracted text sufficiently for chunking and retrieval.
FR-12	Chunking methods	The system shall implement fixed-size, recursive, and semantic chunking methods.
FR-13	Chunk overlap	Each chunking method shall support configurable overlap between neighboring chunks.
FR-14	Chunk metadata	Each chunk shall retain documentId, documentName, chunkId, chunkIndex, and chunkingMethod; pageNumber shall be included when available.
FR-15	Embedding generation	The system shall generate embeddings for document chunks using the selected Sentence Transformers model.
FR-16	Vector storage and search	The system shall store chunk vectors and payloads in Qdrant and perform vector similarity search using HNSW indexing.
FR-17	Keyword search	The system shall index chunk text with BM25 and retrieve keyword-relevant chunks.
FR-18	Hybrid retrieval	The system shall combine vector and BM25 ranked results using Reciprocal Rank Fusion.
FR-19	Evidence-grounded answers	The system shall provide retrieved context to the LLM and return answers grounded in that context with source references.
FR-20	Insufficient evidence	When the available documents do not provide sufficient evidence, the system shall clearly say that the answer was not found in the available university documents.
FR-21	Redis answer caching	The system shall check Redis before retrieval and generation, return cached answers and sources on a hit, and cache successful answers and sources on a miss.
FR-22	Cache invalidation	The system shall prevent stale cached answers from being reused after relevant document changes, using knowledge-base versioning or cache invalidation.
FR-23	Cache analytics	The system shall record and display basic cache metrics, including hits, misses, and hit rate.
FR-24	Document lifecycle	The system shall update or remove corresponding Qdrant points and BM25 entries when a document is updated, deleted, or reindexed.

8. Main System Workflows

8.1 Document Ingestion Workflow

- **1.** An admin uploads a PDF, DOCX, or TXT file.

- **2.** FastAPI validates the file type and size.
- **3.** The original file is stored locally and a document record is created in MySQL.
- **4.** The backend extracts and cleans the document text.
- **5.** The chunking methods split the text into chunks. Each method supports configurable overlap; the methods are evaluated and one is selected as the default for normal ingestion.
- **6.** Metadata is attached to every chunk.
- **7.** The Sentence Transformer model generates an embedding for each chunk.
- **8.** Chunk text, vectors, and metadata are stored in Qdrant.
- **9.** Qdrant uses HNSW for vector search; only useful payload indexes are created.
- **10.** The chunk text is added to the BM25 index.
- **11.** MySQL document status is updated to completed or failed.
- **12.** When an existing document is updated or deleted, old Qdrant points and BM25 entries are removed or replaced, and the knowledge-base version is updated or relevant Redis cache entries are invalidated.

8.2 Question-Answering Workflow

- **1.** An authenticated user submits a question.
- **2.** FastAPI identifies the user and determines the applicable access scope.
- **3.** The backend normalizes the question and builds a Redis key using the knowledge-base version, access scope, and question hash.
- **4.** The backend checks Redis before performing embedding or retrieval.
- **5. Cache hit:** return the cached answer and source references. Skip question embedding, Qdrant, BM25, RRF, and LLM generation. Save the current user's chat interaction in MySQL and record a cache hit.
- **6. Cache miss:** generate the question embedding, search Qdrant, and run BM25 keyword search.
- **7.** RRF combines the vector and keyword result rankings into a final set of context chunks.
- **8.** The backend checks whether the retrieved evidence is sufficient. If not, it returns a clear not-found response and does not cache it as a successful answer.
- **9.** The LLM generates an answer grounded in the retrieved context and includes source references.
- **10.** The backend returns the answer and sources, stores the successful result in Redis with the configured TTL, and saves the chat interaction in MySQL.
- **11.** The backend records cache and performance metrics.

Cache keys must respect access scope. Answers retrieved from restricted documents must not be shared with users who do not have permission to access those documents.

9. Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	Redis hits should return without repeating embedding, retrieval, or LLM generation. Retrieval and response times should be measured during testing.
NFR-02	Security	Passwords must be hashed; protected routes must validate JWTs and enforce backend role permissions.
NFR-03	Data privacy	Chat history and restricted document content must only be accessible to authorized users.
NFR-04	Reliability	If Redis is unavailable, the system should continue using the normal RAG pipeline where possible.
NFR-05	Answer quality	Answers should be grounded in retrieved university documents and include accurate source references. The system should avoid unsupported answers.
NFR-06	Maintainability	Use a clear modular project structure and keep code simple, readable, and easy to explain.
NFR-07	Scalability	Use Qdrant indexing and Redis caching to support efficient retrieval and reduce repeated work. Avoid premature infrastructure complexity.
NFR-08	Usability	Provide a clear responsive interface with understandable navigation, loading states, and error messages.

10. Testing and Evaluation

10.1 Functional Testing

- Test registration, login, JWT validation, and role restrictions.
- Verify that guests are redirected to login before accessing chat.
- Test upload and extraction for PDF, DOCX, and TXT files, including invalid file types.
- Verify that all three chunking methods produce chunks with overlap and correct metadata.
- Verify Qdrant storage/search, BM25 retrieval, and RRF result combination.
- Test Redis cache miss, cache hit, TTL expiry, invalidation after document changes, and graceful behavior when Redis is unavailable.
- Verify that answers include correct source references and that insufficient-evidence questions are handled clearly.

10.2 Chunking Evaluation

Compare fixed-size, recursive, and semantic chunking on the same document corpus, embedding model, retrieval configuration, and fixed question set. Disable Redis during this evaluation so cached responses do not affect retrieval measurements.

Use a modest evaluation set of approximately 20–30 questions, including answerable questions from one document, questions requiring multiple documents, exact names/dates/identifiers, questions absent from the documents, and unrelated questions.

Compare chunk count and size, retrieval quality (Recall@k and MRR where feasible), answer correctness, source/citation accuracy, and processing or retrieval time. Select the best-performing method as the default while keeping all three methods available for comparison.

10.3 Cache Evaluation

Run the same questions with caching enabled and disabled. Compare response latency, cache hit rate, and the number of LLM calls avoided. Report measured results only; do not invent performance improvements.

Cache hit rate = hits / (hits + misses) × 100%.

10.4 Retrieval and Answer Quality

Inspect whether relevant passages appear in the top retrieved results, whether the generated answer is supported by those passages, and whether the citations point to the correct document and page/chunk where available.

11. Initial Project Structure

```
campusmind-ai/
■■■ frontend/
■ ■■■ (React application)
■■■ backend/
■ ■■■ app/
■ ■ ■■■ main.py
■ ■ ■■■ api/
■ ■ ■■■ core/
■ ■ ■■■ models/
■ ■ ■■■ schemas/
■ ■ ■■■ services/
■ ■ ■■■ repositories/
■ ■ ■■■ utils/
■ ■■■ requirements.txt
■■■ storage/
■ ■■■ uploads/
■■■ docker-compose.yml
■■■ .env.example
■■■ .gitignore
■■■ README.md
```

The structure is a starting point, not a requirement to create empty folders. Add modules when they are needed. Keep the backend as one modular application.

12. Constraints and Assumptions

- The initial version uses one FastAPI backend and local development services; production deployment details can be decided later.
- An external LLM API is used for answer generation. The provider and model will be selected separately.
- The selected embedding model is BAAI/bge-small-en-v1.5; its vector dimension must be used consistently in Qdrant.
- Uploaded documents are assumed to contain extractable text. Scanned PDFs may require OCR, which is not part of the initial scope unless needed.
- Document permissions and access scopes must be applied consistently in retrieval and cache keys if restricted documents are introduced.
- Redis is an optimization, not the only source of truth. The application must still be able to answer through the normal pipeline when Redis is unavailable.
- Qdrant and Redis run locally through Docker during development; MySQL is managed locally.

13. Future Improvements

- Add OCR support for scanned documents if required.
- Add more detailed document-level permissions if the university requires department-specific or private documents.
- Consider reranking only after evaluating the quality of the initial hybrid retrieval pipeline.
- Consider cloud deployment and object storage when moving beyond local development.

- Improve evaluation datasets and retrieval parameters based on measured results.

These improvements are not required for the first implementation and should not delay the core system.

14. Expected Deliverables

- Working React frontend with the six agreed pages and role-aware navigation.
- FastAPI backend with authentication, document ingestion, RAG question answering, and chat history.
- MySQL schema and database setup instructions.
- Qdrant collection with vector storage, HNSW search, and required payload indexes.
- BM25 keyword index and RRF hybrid retrieval implementation.
- Three chunking methods with configurable overlap and evaluation results.
- Redis answer caching with cache hit/miss metrics and invalidation behavior.
- README containing setup instructions, architecture overview, environment configuration, and how to run the application.
- Test evidence and a short evaluation comparing chunking methods and caching enabled versus disabled.

15. Development Guidelines

- Follow this SRS as the agreed scope for the initial implementation.
- Implement one feature at a time and test it before moving to the next feature.
- Keep the code simple, clean, modular, and easy to explain during a presentation.
- Do not add unnecessary libraries, services, abstractions, or advanced infrastructure.
- Use clear names and keep responsibilities separated without creating excessive layers.
- Do not claim performance gains without measuring them.
- Preserve source metadata throughout ingestion, retrieval, answer generation, and chat display.
- Keep BM25 hybrid retrieval, Qdrant vector indexing, Redis answer caching, and all three overlap-enabled chunking methods in the core implementation.

End of SRS — Version 1.2